

Experiment 6

Student Name: Prabhakar Kr. Jha

Branch: CSE

Semester: 6th

Subject Name: Full Stack Development – II

UID: 23BCS12284

Section/Group: KRG 3-B

Date of Performance: 03/03/2026

Subject Code: 23CSH-309

1. **Aim:** To design and implement a **Spring Boot REST API using Layered Architecture**, applying **Inversion of Control (IoC)** and **Dependency Injection (DI)** to build a structured backend for the **HealthHub application**, including CRUD operations, DTO mapping, validation, and global exception handling.

2. Objective:

- Understand the role of **Spring Boot in backend development**
- Configure and run a **basic Spring Boot application**
- Apply **IoC (Inversion of Control)** and **Dependency Injection**
- Use Spring annotations such as **@Autowired**, **@Service**, and **@Repository**
- Implement **Layered Architecture (Controller – Service – Repository)**
- Develop **RESTful APIs using Spring Boot**
- Perform **CRUD operations (Create, Read, Update, Delete)** using HTTP methods
- Implement **DTO mapping** to separate internal data models from API responses
- Apply **validation using Jakarta Validation annotations** like **@NotNull**, **@Email**
- Implement **Global Exception Handling using @ControllerAdvice**
- Build backend services for **HealthHub patient management system**.

3. Implementation / Code:

▪ **Tools & Technologies Used:-**

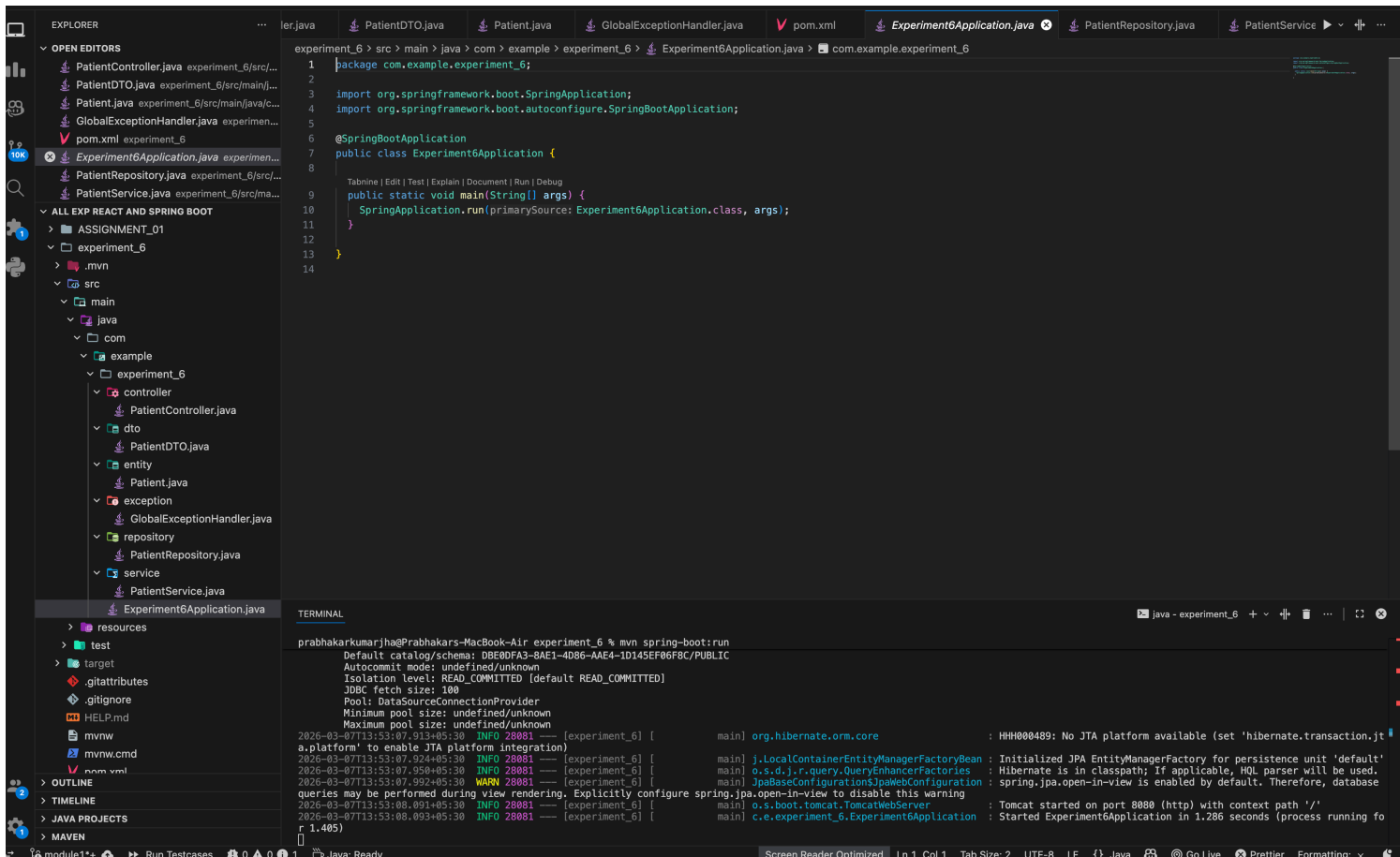
- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Maven
- VS Code
- Postman (API testing)
- H2 Database

▪ **Implementation Description:-**

- A **Spring Boot** application named **HealthHub** is created to manage patient data.
- The project follows **Layered Architecture**, separating logic into different layers:

- **Controller Layer**
Handles HTTP requests and responses using REST APIs.
- **Service Layer**
Contains business logic for managing patients.
- **Repository Layer**
Communicates with the database using Spring Data JPA.
- **Entity Layer**
Represents database tables.
- **DTO Layer**
Transfers patient data between API and application.
- **Exception Layer**
Handles errors globally using @ControllerAdvice.

▪ Project Structure:-



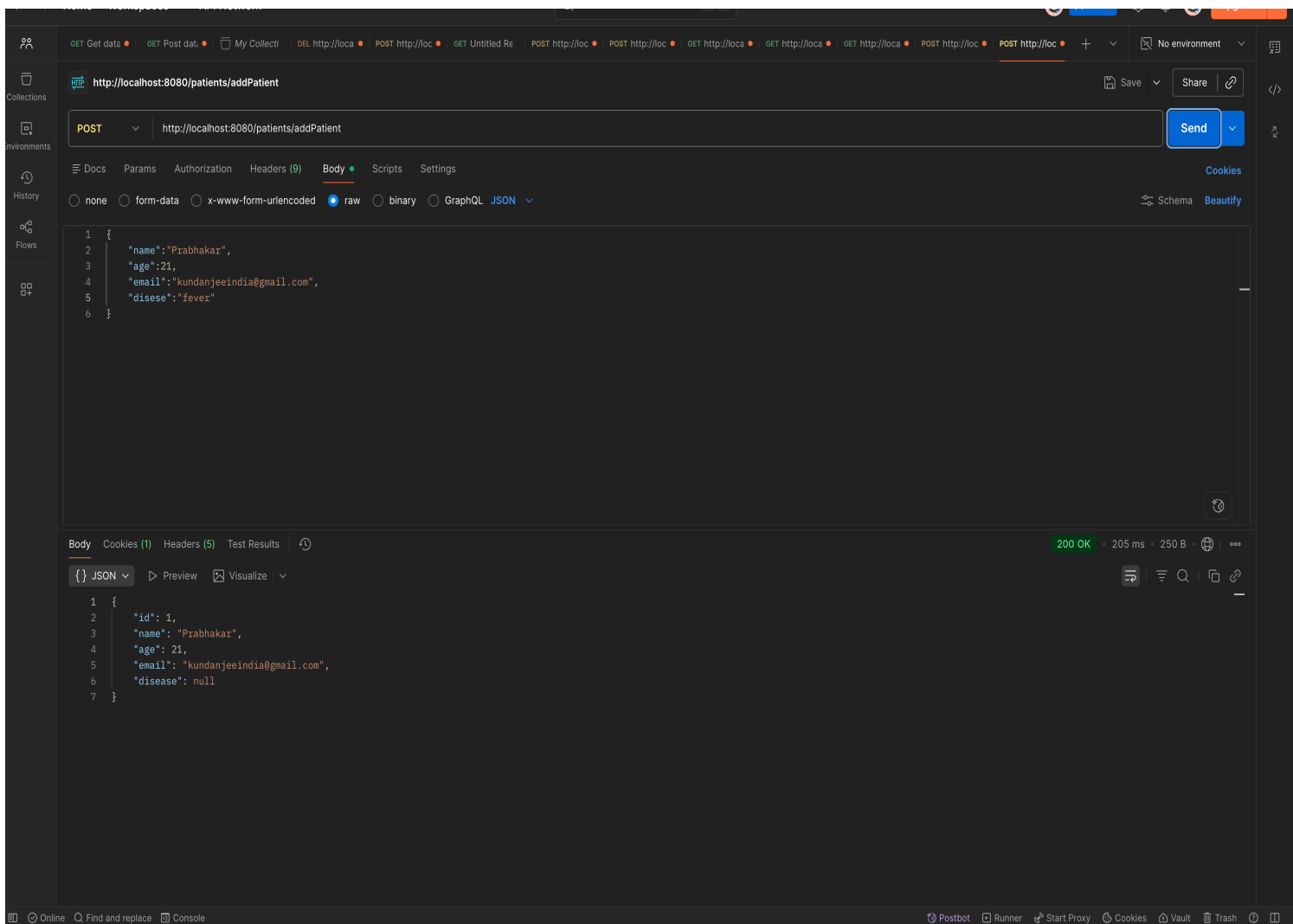
The screenshot displays an IDE with the following components:

- EXPLORER:** Shows the project structure. The 'src/main/java/com/example/experiment_6' package contains:
 - controller: PatientController.java
 - dto: PatientDTO.java
 - entity: Patient.java
 - exception: GlobalExceptionHandler.java
 - repository: PatientRepository.java
 - service: PatientService.java
 - Experiment6Application.java
- Experiment6Application.java:** The main application class, which is a Spring Boot application. It contains the following code:


```
1 package com.example.experiment_6;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class Experiment6Application {
8
9     Tabnine | Edit | Test | Explain | Document | Run | Debug
10     public static void main(String[] args) {
11         SpringApplication.run(primarySource: Experiment6Application.class, args);
12     }
13 }
14
```
- TERMINAL:** Shows the output of the command 'mvn spring-boot:run'. The output includes:
 - Default catalog/schema: DBE0DFA3-8AE1-4D86-AAE4-1D145EF86F8C/PUBLIC
 - Autocommit mode: undefined/unknown
 - Isolation level: READ_COMMITTED [default READ_COMMITTED]
 - JDBC fetch size: 100
 - Pool: DataSourceConnectionProvider
 - Minimum pool size: undefined/unknown
 - Maximum pool size: undefined/unknown
 - 2026-03-07T13:53:07.913+05:30 INFO 28081 [experiment_6] [main] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
 - 2026-03-07T13:53:07.924+05:30 INFO 28081 [experiment_6] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
 - 2026-03-07T13:53:07.950+05:30 INFO 28081 [experiment_6] [main] o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
 - 2026-03-07T13:53:07.992+05:30 WARN 28081 [experiment_6] [main] JpaBaseConfiguration\$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
 - 2026-03-07T13:53:08.091+05:30 INFO 28081 [experiment_6] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
 - 2026-03-07T13:53:08.093+05:30 INFO 28081 [experiment_6] [main] c.e.experiment_6.Experiment6Application : Started Experiment6Application in 1.286 seconds (process running for 1.405s)

4. Output:

- Spring Boot application started successfully on **localhost:8080**
- POST API successfully added patient records
- GET API returned the list of patients stored in the database
- GET by ID API returned specific patient details
- DELETE API successfully removed patient data
- Postman confirmed successful API responses with **Status 200 OK**
- Layered architecture ensured separation of concerns and clean backend design



The screenshot shows the Postman interface for a REST client. The top bar displays the URL `http://localhost:8080/patients/addPatient` and the method `POST`. The request body is a JSON object:

```
1 {
2   "name": "Prabhakar",
3   "age": 21,
4   "email": "kundanjeeindia@gmail.com",
5   "disease": "fever"
6 }
```

The response is shown in the bottom panel, indicating a `200 OK` status with a response time of `205 ms` and a body size of `250 B`. The response body is a JSON object:

```
1 {
2   "id": 1,
3   "name": "Prabhakar",
4   "age": 21,
5   "email": "kundanjeeindia@gmail.com",
6   "disease": null
7 }
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

GET http://localhost:8080/patients/getPatients

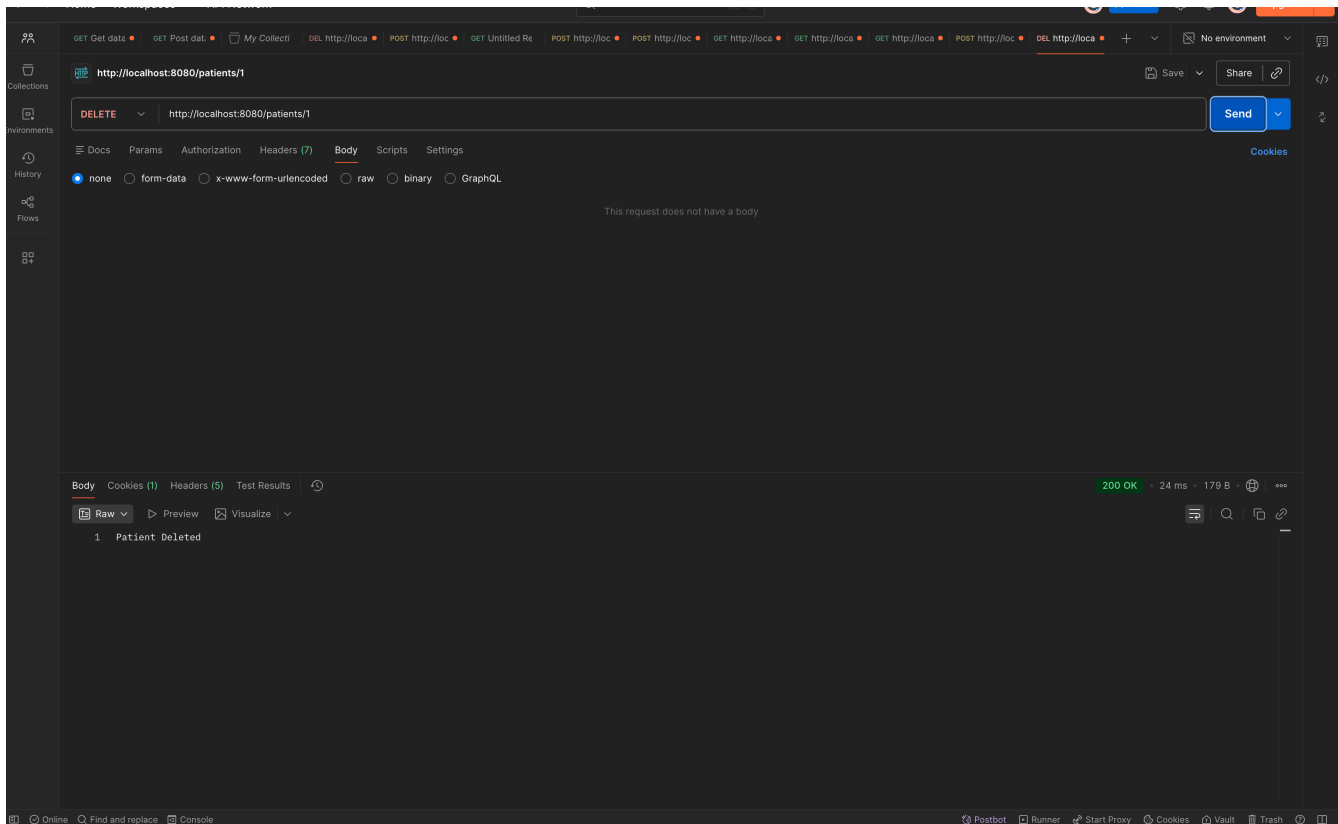
200 OK · 23 ms · 252 B

```
{
  "id": 1,
  "name": "Prabhakar",
  "age": 21,
  "email": "kundanjeeindia@gmail.com",
  "disease": null
}
```

GET http://localhost:8080/patients/1

200 OK · 24 ms · 250 B

```
{
  "id": 1,
  "name": "Prabhakar",
  "age": 21,
  "email": "kundanjeeindia@gmail.com",
  "disease": null
}
```



5. Learning Outcomes (What I Have Learnt):

- Understanding of **Spring Boot** backend development
- Implementation of **Layered Architecture (Controller, Service, Repository)**
- Usage of **Dependency Injection** in Spring Boot
- Development of **RESTful APIs** using HTTP methods
- Implementation of **CRUD** operations in backend applications
- Understanding of **DTO mapping** for API communication
- Applying **validation annotations** for data integrity
- Handling errors globally using **Spring Exception Handling**
- Testing APIs using **Postman**